LAB 4 – VIBE CODING AI CHAT HISTORY

Project: Defender Game

Student Name and SRN: SKANDA M L - PES1UG24CS457
GitHub Repository: https://github.com/skandaml21/28_defender
Programming Language: Python
Game Library: Pygame
Development Environment: Windows PowerShell
AI Assistant: ChatGPT

---

1. Objective

The objective of this lab was to understand, modify, test, and improve an existing Python-based Defender game using AI-assisted coding.

The assigned tasks included:

1. Forking and cloning the provided GitHub repository.
2. Understanding the project and running the original game.
3. Fixing the radar world-position mapping bug.
4. Implementing wave-based sky colors.
5. Adding a humanoid rescue notification.
6. Implementing a bonus-life score threshold.
7. Testing the modified game and committing changes separately.
8. Pushing the final code to the personal GitHub repository.
9. Recording before-and-after gameplay videos.

2. Initial Project Setup

Prompt 1:

Guide me step by step through the Lab 4 Vibe Coding assignment. I am using Windows PowerShell and want to complete the tasks in the provided Defender repository.

AI Assistant response:

The recommended workflow was to fork the original repository, clone the personal fork, inspect the README and source code, run the game, implement the required changes, test the code, commit each task separately, and push the final changes to GitHub.

Original repository:

https://github.com/SETAPESU26/28_defender

Personal repository:

https://github.com/skandaml21/28_defender

The game was developed using Python and Pygame.

3. Understanding the Required Tasks

Prompt 2:

Explain the tasks that need to be implemented in the Defender game.

AI Assistant response:

The project required four main improvements.

Task 1: Fix the Radar Bug

The radar should display the player's and other objects' positions using their absolute world coordinates. The original implementation incorrectly used a player-relative screen position.

The mapping was corrected to use the object's world coordinate:

rx = (x % WORLD_W) / WORLD_W * VIEW_W

This ensures that the radar position is calculated from the world position rather than the player's relative screen position.

Task 2: Implement Wave-Based Sky Colors

The `sky_color(wave)` function was implemented to return RGB color values according to the current wave.

The function uses different colors for different wave ranges and returns `None` for the default sky in the first wave.

Task 3: Add a Humanoid Rescue Notification

The `on_humanoid_rescued(humanoid)` function was implemented to provide feedback when the player rescues a falling humanoid.

The implementation displays a green `+500 RESCUE!` notification for a short duration after a successful rescue.

Task 4: Implement the Bonus-Life Threshold

The `bonus_life_threshold()` function was implemented to return an integer score threshold.

The selected threshold was 10,000 points:

```
def bonus_life_threshold():
    return 10000
```

The existing bonus-life tracking mechanism was retained.

4. Task 1 – Radar Bug Fix

Prompt 3:

Help me fix the radar world-position mapping bug in `game.py`.

AI Assistant response:

The radar mapping was changed to use the absolute horizontal world coordinate. This avoids calculating the radar location from a player-relative screen coordinate.

Implementation:

rx = (x % WORLD_W) / WORLD_W * VIEW_W

Testing:

The Python file was checked for syntax errors.

Git commit:

`940d862 Fix radar world-position mapping`

5. Task 2 – Wave-Based Sky Colors

Prompt 4:

Help me implement the `sky_color(wave)` function with different sky colors depending on the current wave.

AI Assistant response:

The function was implemented with different RGB values for different wave ranges.

Implementation:

```
def sky_color(wave):
    """Return an (r, g, b) sky colour for the current wave, or None for the default."""
    if wave <= 1:
        return None
    elif wave <= 3:
        return (8, 8, 35)
    elif wave <= 5:
        return (20, 8, 45)
    elif wave <= 7:
        return (35, 8, 35)
    else:
        return (45, 12, 20)
```

Expected result:

The sky changes color as the player progresses through different waves, while the first wave retains the default sky.

Git commit:

`112cae5 Add wave-based sky colors`

6. Task 3 – Humanoid Rescue Notification

Prompt 5:

Help me implement the humanoid rescue callback so that the player receives visible feedback after rescuing a humanoid.

AI Assistant response:

A rescue notification was added and displayed temporarily on the game screen.

The implementation initializes a notification message and timer, updates the timer during the game loop, and renders the message while the timer remains active.

Implementation details:

- The rescued humanoid is set to the grounded state.
- The player receives 500 points.
- The rescue callback returns the message `+500 RESCUE!`.
- A timer controls how long the notification remains visible.
- The notification is rendered in green near the top of the gameplay area.

Git commit:

`a2f2067 Add humanoid rescue reward notification`

7. Task 4 – Bonus-Life Score Threshold

Prompt 6:

Help me implement the `bonus_life_threshold()` function.

AI Assistant response:

The function was implemented to return a threshold of 10,000 points.

Implementation:

```
def bonus_life_threshold():
    return 10000
```

The existing bonus-life tracking logic was retained to avoid repeatedly awarding a bonus life for the same threshold.

Git commit:

`2cfc7d3 Add bonus-life score threshold`

8. Testing and Verification

Prompt 7:

How can I check whether the modified Python code is valid and the game runs correctly?

AI Assistant response:

The following commands were used in Windows PowerShell.

Syntax verification

python -m py_compile game.py

The command completed without a syntax error.

Run the game

python game.py

Pygame started successfully, and the game window opened.

The game was also used to prepare the before-and-after gameplay recordings.

9. Git Commit History

The changes were committed separately to make the development process traceable.

Commit	Description
`940d862`	Fix radar world-position mapping
`112cae5`	Add wave-based sky colors
`a2f2067`	Add humanoid rescue reward notification
`2cfc7d3`	Add bonus-life score threshold

The commits were visible in the local Git history.

The following command was used to inspect the recent commits:

git log --oneline -4

10. GitHub Push Issue and Resolution

Prompt 8:

I received a permission-denied error when pushing the code to GitHub. How do I fix it?

Observed error:

remote: Permission to skandaml21/28_defender.git denied to Varshitha7-S.
fatal: unable to access

'https://github.com/skandaml21/28_defender.git/':
The requested URL returned error: 403

AI Assistant response:

The remote URL was correct, but Git was authenticating with the wrong GitHub account.

The incorrect cached GitHub credential was removed through Windows Credential Manager.

The push was then retried.

Push command:

git push origin main

Successful output:

To https://github.com/skandaml21/28_defender.git
031f1ee..2cfc7d3 main -> main

This confirmed that the local changes were successfully pushed to the personal repository.

Personal repository:

https://github.com/skandaml21/28_defender

11. Final Deliverables

The following deliverables were planned for submission:

1. **Before video:** Approximately 10 seconds of gameplay before modifications.
2. **After video:** Approximately 10 seconds of gameplay after modifications.
3. **Updated source code:** The modified `game.py` file.
4. **Chat history:** This document, exported as a PDF.
5. **GitHub repository:** The personal repository containing the committed changes.

The planned submission folder was:

```
Lab-4/
├── before.mp4
├── after.mp4
├── game.py
└── Chat_History.pdf
```

12. Conclusion

The Defender game was enhanced through AI-assisted development. The radar mapping issue was corrected, wave-based sky colors were introduced, a humanoid rescue notification was implemented, and the bonus-life score threshold was configured.

The modified Python file passed syntax verification, the game launched successfully, and all four task commits were pushed to the personal GitHub repository.

This lab demonstrated the use of AI assistance for code understanding, debugging, feature implementation, testing, version control, and project submission.